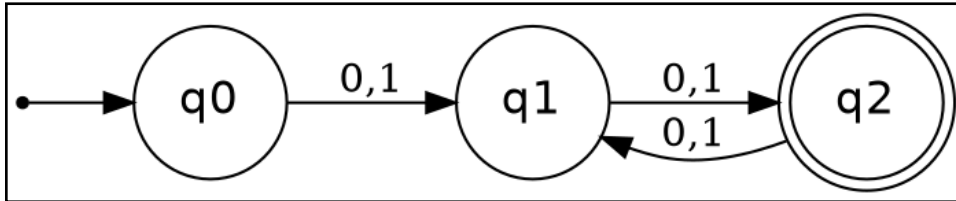


CO2_AT2 – Design Task: Theory of Computation

Question 1

Design the regular expression for the language represented by the given automata.

Given Automata



The diagram shows states q_0 (start), q_1 , and q_2 (final, double circle), with the following transitions:

- $q_0 \rightarrow q_1$ on input 0 or 1
- $q_1 \rightarrow q_2$ on input 0 or 1
- $q_2 \rightarrow q_1$ on input 0 or 1

Analysis

From q_0 , exactly one symbol is consumed to reach q_1 . From q_1 the machine moves to q_2 (accepting) after one more symbol, and from q_2 it moves back to q_1 after one more symbol, alternating with period 2 for every symbol consumed thereafter.

Hence the automaton is in the accepting state q_2 exactly when the total number of input symbols read is even and at least 2. In other words, the language accepted is the set of all strings over $\{0,1\}$ of even length (excluding the empty string).

$$L = \{ w \in \{0,1\}^* : |w| \text{ is even and } |w| \geq 2 \}$$

Answer – Regular Expression

$$R = ((0+1)(0+1))^+ \quad [\text{equivalently: } (0+1)(0+1)((0+1)(0+1))^*]$$

This expression pairs up input symbols two at a time, one-or-more times, which exactly matches strings of even, non-zero length.

Question 2

Consider the following grammar:

$$S \rightarrow aSc \mid T \mid U \mid \varepsilon$$

$$T \rightarrow bTc \mid \varepsilon$$

$$U \rightarrow aUb \mid \varepsilon$$

(A) Language defined by the grammar

The rule $S \rightarrow aSc$ can be applied $n \geq 0$ times, unwinding to $S \Rightarrow^* a^n S c^n$, after which S is replaced by ε , or by T , or by U .

- $T \rightarrow bTc \mid \varepsilon$ generates $b^m c^m$ for $m \geq 0$.
- $U \rightarrow aUb \mid \varepsilon$ generates $a^k b^k$ for $k \geq 0$.

Substituting each branch back into $a^n S c^n$ gives three cases:

- Via ε : $a^n c^n$
- Via T : $a^n (b^m c^m) c^n = a^n b^m c^{(n+m)}$
- Via U : $a^n (a^k b^k) c^n = a^{(n+k)} b^k c^n$

Combining all cases, every derived string has the form $a^i b^j c^k$ (a's, then b's, then c's) where either the number of c's equals the sum of a's and b's, or the number of a's equals the sum of b's and c's:

$$L = \{ a^i b^j c^k \mid i, j, k \geq 0, \text{ and } (k = i+j) \text{ or } (i = j+k) \}$$

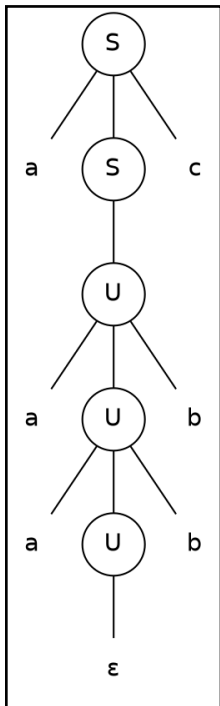
(B) Derivation tree for aaabbc

For aaabbc: $i = 3, j = 2, k = 1$. Check: $k = i+j \rightarrow 1 = 5$ (false). $i = j+k \rightarrow 3 = 2+1 = 3$ (true). So this string is generated via the U-branch, with $n = 1$ (outer a...c wrap) and $k = 2$ (two applications of $U \rightarrow aUb$).

Derivation:

$S \Rightarrow aSc \Rightarrow aUc \Rightarrow a(aUb)c \Rightarrow aa(aUb)bc \Rightarrow aa(ab)bc \Rightarrow aaabbc$

(using $S \rightarrow aSc$, then $S \rightarrow U$, then $U \rightarrow aUb$ twice, then $U \rightarrow \epsilon$)



(C) Is the grammar ambiguous?

A grammar is ambiguous if some string has two distinct derivation trees (or leftmost derivations).

Consider the string "ac" ($n = 1$, with the inner part empty). It can be derived in three different ways:

- $S \Rightarrow aSc \Rightarrow a(\epsilon)c = ac$, using $S \rightarrow \epsilon$ directly
- $S \Rightarrow aSc \Rightarrow aTc \Rightarrow a(\epsilon)c = ac$, using $S \rightarrow T$, then $T \rightarrow \epsilon$

- $S \Rightarrow aSc \Rightarrow aUc \Rightarrow a(\epsilon)c = ac$, using $S \rightarrow U$, then $U \rightarrow \epsilon$

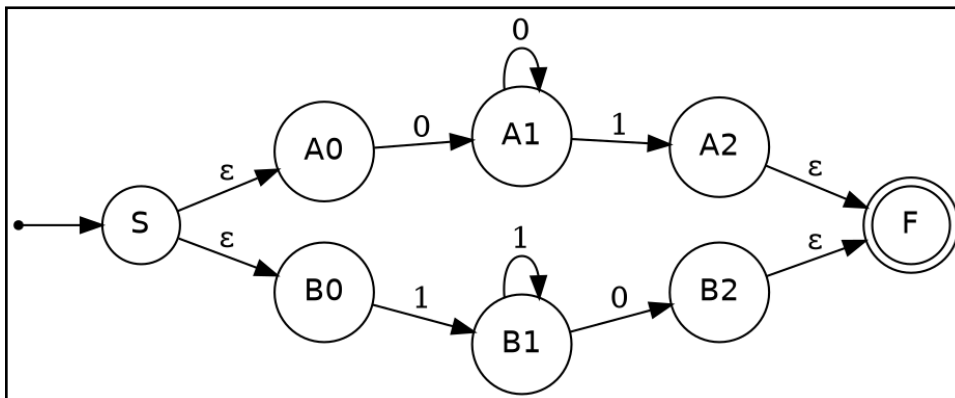
These are three distinct parse trees producing the identical string "ac". Since a string has more than one derivation tree, the grammar is ambiguous.

Question 3

Design a non-deterministic finite automaton with ϵ -transitions to represent the language denoted by the regular expression:

$$00^*1 + 11^*0$$

This is the union of two sub-languages: $L1 = 00^*1$ (one or more 0's followed by a single 1) and $L2 = 11^*0$ (one or more 1's followed by a single 0). Using Thompson's construction, a new start state S is connected by ϵ -transitions to the start of each branch, and both branches are joined by ϵ -transitions into a common final state F.



Formal description:

- $S \xrightarrow{\epsilon} A0$ and $S \xrightarrow{\epsilon} B0$ (branch selection)
- Branch 1 (00^*1): $A0 \xrightarrow{0} A1$, $A1 \xrightarrow{0} A1$ (self-loop), $A1 \xrightarrow{1} A2$, $A2 \xrightarrow{\epsilon} F$
- Branch 2 (11^*0): $B0 \xrightarrow{1} B1$, $B1 \xrightarrow{1} B1$ (self-loop), $B1 \xrightarrow{0} B2$, $B2 \xrightarrow{\epsilon} F$

F is the sole accepting (final) state. Any string matching 00^*1 or 11^*0 drives the NFA from S to F along one of the two branches.

Question 4

(i) If L_1 and L_2 are regular, then $L_1 + L_2$ (union) is also regular

Proof: Since L_1 and L_2 are regular, by definition there exist regular expressions r_1 and r_2 such that $L(r_1) = L_1$ and $L(r_2) = L_2$. (Equivalently, there exist DFAs/NFAs M_1 and M_2 accepting L_1 and L_2 .)

Regular expression argument: $r_1 + r_2$ is itself a valid regular expression (union is one of the basic regex operations), and by definition $L(r_1 + r_2) = L(r_1) \cup L(r_2) = L_1 \cup L_2$. Since $L_1 \cup L_2$ is described by a regular expression, it is regular.

Automaton argument (equivalent): Construct a new NFA M by adding a new start state q_0' with ϵ -transitions to the start states of M_1 and M_2 . M accepts w if and only if M_1 accepts w or M_2 accepts w , so $L(M) = L_1 \cup L_2$. Since an NFA exists for $L_1 \cup L_2$, it is regular (NFAs and DFAs are equivalent in expressive power).

Hence regular languages are closed under union, so $L_1 + L_2$ is regular.

(ii) Is the language $\{ a^p \mid p \text{ is prime} \}$ regular? Justify.

Claim: No, this language is NOT regular.

Proof (by the Pumping Lemma for regular languages): Suppose, for contradiction, that $L = \{ a^p : p \text{ is prime} \}$ is regular. Then there exists a pumping length n such that every string $w \in L$ with $|w| \geq n$ can be split as $w = xyz$, where $|xy| \leq n$, $|y| \geq 1$, and $xy^i z \in L$ for every $i \geq 0$.

Since there are infinitely many primes, choose a prime $p \geq n + 2$, and let $w = a^p$, so $|w| = p \geq n$. By the pumping lemma, $w = xyz$ with $|y| = k$, where $1 \leq k \leq n$.

Then for every $i \geq 0$, the string $xy^i z$ has length $p + (i - 1)k$, and this length must also be prime for $xy^i z \in L$ to hold.

Choose $i = p + 1$. Then:

$$|xy^{(p+1)}z| = p + (p+1-1)k = p + pk = p(1 + k)$$

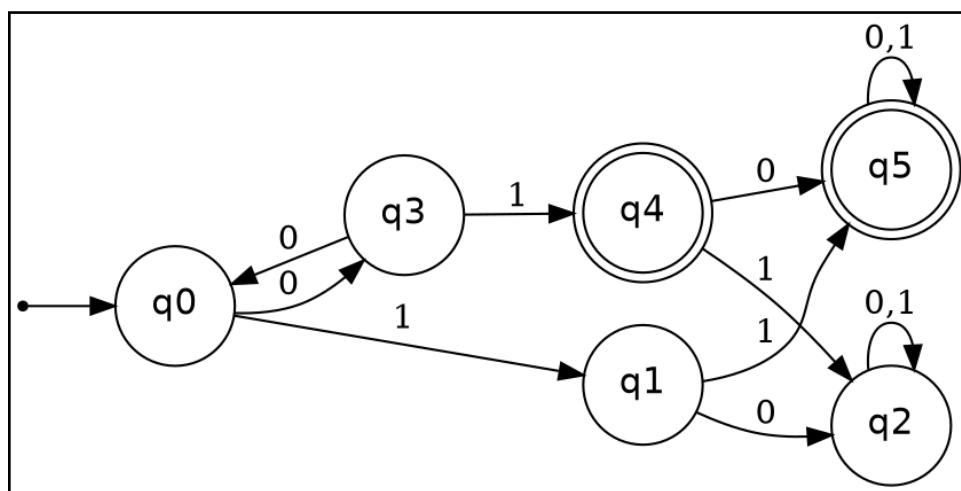
Since $p \geq 2$ and $k \geq 1$, both factors p and $(1+k)$ are at least 2, so $p(1+k)$ is a composite number, not prime. Therefore $xy^{(p+1)}z \notin L$, contradicting the pumping lemma's guarantee that $xy^i z \in L$ for all $i \geq 0$.

This contradiction shows that our assumption is false. Hence $L = \{ a^p \mid p \text{ is prime} \}$ is not regular.

Question 5

Design minimized automata. Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table, and mark initial and final states.

Given DFA



States: $\{q0, q1, q2, q3, q4, q5\}$; Start state: $q0$; Final states: $\{q4, q5\}$

State	0	1
q0	q3	q1
q1	q2	q5
q2	q2	q2
q3	q0	q4

q4	q5	q2
q5	q5	q5

Step 1 — Initial Partition

Split states into Final (F) and Non-Final (NF):

- $F = \{ q4, q5 \}$
- $NF = \{ q0, q1, q2, q3 \}$

Step 2 — Round 1 refinement

For each state, find the group (F / NF) reached on input 0 and on input 1:

State	0	1
q0	q3 (NF)	q1 (NF)
q1	q2 (NF)	q5 (F)
q2	q2 (NF)	q2 (NF)
q3	q0 (NF)	q4 (F)
q4	q5 (F)	q2 (NF)
q5	q5 (F)	q5 (F)

Grouping states with identical (0,1) signatures gives the refined partition:

- $\{ q0, q2 \}$ — signature (NF, NF)
- $\{ q1, q3 \}$ — signature (NF, F)
- $\{ q4 \}$ — signature (F, NF)
- $\{ q5 \}$ — signature (F, F)

Step 3 — Round 2 refinement

Re-check each group using the Round-1 partition labels $A = \{q0, q2\}$, $B = \{q1, q3\}$, $C = \{q4\}$, $D = \{q5\}$:

- q0: $0 \rightarrow q3 (B), 1 \rightarrow q1 (B) \rightarrow \text{signature } (B, B)$
- q2: $0 \rightarrow q2 (A), 1 \rightarrow q2 (A) \rightarrow \text{signature } (A, A)$

q_0 and q_2 now have different signatures, so they are NOT equivalent — split $\{q_0, q_2\}$ into $\{q_0\}$ and $\{q_2\}$.

- q_1 : $0 \rightarrow q_2$ (A), $1 \rightarrow q_5$ (D) $\rightarrow$ signature (A, D)
- q_3 : $0 \rightarrow q_0$ (A), $1 \rightarrow q_4$ (C) $\rightarrow$ signature (A, C)

q_1 and q_3 also have different signatures, so they are NOT equivalent — split $\{q_1, q_3\}$ into $\{q_1\}$ and $\{q_3\}$.

Refined partition after Round 2: $\{q_0\}$, $\{q_1\}$, $\{q_2\}$, $\{q_3\}$, $\{q_4\}$, $\{q_5\}$ — all singleton groups.

Step 4 — Termination

Since every group is now a singleton, no further refinement is possible. No pair of states is equivalent.

Conclusion

No states can be merged, so the given DFA is already minimal — it has no redundant states. The minimized automaton is identical to the original 6-state DFA shown above.

Minimized DFA specification:

- States: $\{q_0, q_1, q_2, q_3, q_4, q_5\}$
- Start state: q_0
- Final states: $\{q_4, q_5\}$

State	0	1
$\rightarrow q_0$	q_3	q_1
q_1	q_2	q_5
q_2	q_2	q_2
q_3	q_0	q_4
$* q_4$	q_5	q_2
$* q_5$	q_5	q_5

($\rightarrow$ marks the initial state, $*$ marks final states)

